Code walkthrough

tb_register_file.v

Testbench: Register File

What it does	Checks reset, write and read (11 checks)
Where it works	module-level test
Size	104 lines, explained in 11 small parts
Modules in this file	tb_register_file
Calls (uses)	register_file.v (module register_file)
Called by	nobody - this is a testbench (top of simulation)

This testbench checks the register file on its own: reset, writing each register, reading on both ports, and that nothing is written when we = 0.

Map of the file

Part	Lines	What this part is
1	1-11	Header
2	12-24	Signals and clock
3	26-28	The design under test
4	30-44	Task: check
5	46-55	Task: write_reg
6	57-67	Test 1: reset
7	69-73	Test 2: write all four registers
8	75-84	Test 3: read back
9	86-90	Test 4: write enable off
10	92-96	Test 5: second reset
11	98-104	Summary

> How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

Part 1 · Header (lines 1-11)

tb_register_file.v lines 1-11

```
 1  `timescale 1ns / 1ps
 2  //
 3  // CONNECTIONS
 4  //   Calls       : register_file.v (module register_file)
 5  //   Called by   : nobody - this is a testbench (top of simulation)
 6  // ================================================================
 7  // FILE: tb_register_file.v
 8  // MODULE-LEVEL TEST 2: the register file on its own.
 9  // Checks: reset clears registers, each register can be written and read,
10  // both read ports work at the same time, and we = 0 blocks writing.
11  // ================================================================
```

- Time unit and a description of what is tested.
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

Part 2 · Signals and clock (lines 12-24)

tb_register_file.v lines 12-24

```
12  module tb_register_file;
13
14      reg        clk = 0;
15      reg        rst = 1;
16      reg  [1:0] ra = 0, rb = 0, wa = 0;
17      reg  [3:0] wd = 0;
18      reg        we = 0;
19      wire [3:0] qa, qb;
20
21      integer errors = 0;
22      integer tests  = 0;
23
24      always #5 clk = ~clk;
```

- Inputs (reg) start at 0; rst starts at 1. qa and qb are the outputs.
- `always #5 clk = ~clk;` flips the clock every 5 ns: a 10 ns clock.

Part 3 · The design under test (lines 26-28)

tb_register_file.v lines 26-28

```
26      // CALLS: module register_file from file register_file.v
27      register_file DUT (.clk(clk), .rst(rst), .ra(ra), .rb(rb),
28                         .wa(wa), .wd(wd), .we(we), .qa(qa), .qb(qb));
```

One register file connected to the testbench signals.

→ This part **calls module** `register_file` from file `register_file.v`. How that module works is explained in `Explained_register_file.pdf`.

Part 4 · Task: check (lines 30-44)

tb_register_file.v lines 30-44

```
30      // compare one 4-bit value
31      task check;
32          input [8*26:1] name;
33          input [3:0]    actual;
34          input [3:0]    expected;
35          begin
36              tests = tests + 1;
37              if (actual == expected)
38                  $display("PASS  %0s = %b", name, actual);
39              else begin
40                  $display("FAIL  %0s = %b (expected %b)", name, actual, expected);
41                  errors = errors + 1;
42              end
43          end
44      endtask
```

Compares an actual value with the expected one and prints PASS or FAIL.

Part 5 · Task: write_reg (lines 46-55)

tb_register_file.v lines 46-55

```
46      // write one register (takes one clock edge)
47      task write_reg;
48          input [1:0] addr;
49          input [3:0] data;
50          begin
51              wa = addr;  wd = data;  we = 1;
52              @(posedge clk);  #1;
53              we = 0;
54          end
55      endtask
```

Sets wa, wd and we = 1, waits for one rising clock edge (the write happens there), then sets we back to 0.

Part 6 · Test 1: reset (lines 57-67)

tb_register_file.v lines 57-67

```
57      initial begin
58          // The tasks 'check' and 'write_reg' used below are defined above
59          // in this same file.
60          $display("\n===== tb_register_file: register file test =====");
61
62          // 1. reset
63          @(posedge clk); #1;
64          rst = 0;
65          ra = 0; rb = 3; #1;
66          check("after reset R0 (port A)", qa, 4'b0000);
67          check("after reset R3 (port B)", qb, 4'b0000);
```

After one clock in reset, releases reset and checks that R0 and R3 read 0000.

Part 7 · Test 2: write all four registers (lines 69-73)

tb_register_file.v lines 69-73

```
69          // 2. write a different value into each register
70          write_reg(2'd0, 4'b0001);
71          write_reg(2'd1, 4'b0101);
72          write_reg(2'd2, 4'b1010);
73          write_reg(2'd3, 4'b1111);
```

Writes four different values: R0 = 0001, R1 = 0101, R2 = 1010, R3 = 1111.

Part 8 · Test 3: read back (lines 75-84)

tb_register_file.v lines 75-84

```
75          // 3. read them back on both ports
76          ra = 0; rb = 1; #1;
77          check("R0 on port A", qa, 4'b0001);
78          check("R1 on port B", qb, 4'b0101);
79          ra = 2; rb = 3; #1;
80          check("R2 on port A", qa, 4'b1010);
81          check("R3 on port B", qb, 4'b1111);
82          ra = 1; rb = 1; #1;
83          check("R1 on both ports (A)", qa, 4'b0101);
84          check("R1 on both ports (B)", qb, 4'b0101);
```

Reads the values on both ports at the same time, including the same register on both ports. #1 waits 1 ns so the combinational read has settled.

Part 9 · Test 4: write enable off (lines 86-90)

tb_register_file.v lines 86-90

```
86          // 4. we = 0 must NOT change a register
87          wa = 2; wd = 4'b0000; we = 0;
88          @(posedge clk); #1;
89          ra = 2; #1;
90          check("R2 unchanged when we=0", qa, 4'b1010);
```

Tries to write 0000 into R2 with we = 0 and checks R2 still holds 1010.

Part 10 · Test 5: second reset (lines 92-96)

tb_register_file.v lines 92-96

```
92          // 5. reset again clears everything
93          rst = 1; @(posedge clk); #1; rst = 0;
94          ra = 3; rb = 2; #1;
95          check("R3 after second reset", qa, 4'b0000);
96          check("R2 after second reset", qb, 4'b0000);
```

Resets again and checks the registers are cleared.

Part 11 · Summary (lines 98-104)

tb_register_file.v lines 98-104

```
 98          if (errors == 0)
 99              $display("tb_register_file PASSED: %0d of %0d checks OK\n", tests, tests);
100          else
101              $display("tb_register_file FAILED: %0d of %0d checks failed\n", errors, tests);
102          $finish;
103      end
104  endmodule
```

Prints PASSED / FAILED and ends the simulation.

> **In short:** 11 checks, all passing.